POLITÉCNICA	**Bases de datos Avanzadas** **Curso 2025-26** **Segunda Práctica - Tema 2** **Técnicas de Optimización de Consultas**	Universidad Politécnica de Madrid ETSI SISTEMAS INFORMÁTICOS

Objetivos

1. Expresar consultas sql de forma eficiente.
2. Optimizar consultas sql mediante el uso de índices y el estudio de los planes de ejecución.
3. Estudiar el impacto de los índices en las actualizaciones.
4. Aplicar técnicas de desnormalización de bases de datos relacionales.

Enunciado

En esta práctica vamos a trabajar con una base de datos del departamento de recursos humanos de una empresa ficticia que tiene datos de hace años sobre sus empleados, departamentos, salarios, puestos de trabajo y localización regional de los puestos. La base de datos de empleados a la que llamaremos **bda_p2** se compone de unas pocas tablas principales muy relacionadas entre sí. En el documento **DiccionarioDatos_bda_p2.pdf** disponible en Moodle se describen dichas tablas.

No deberá realizarse ningún tipo de entrega para esta práctica, eso sí, los conocimientos adquiridos en su desarrollo serán objeto de evaluación en el **examen del primer parcial**, que incluye conjuntamente teoría y práctica, y que tendrá lugar el día **24 de abril a las 18:00 en el Bloque IX**.

Desarrollo de la práctica

La práctica se trabajará en las clases de laboratorio durante cuatro sesiones. Se recomienda que en cada semana el alumno trabaje en los contenidos correspondientes de la práctica, tanto durante la clase presencial como fuera de ella. Apartado 1. Creación de la base de datos.

a) Crear la base de datos de la práctica de nombre **bda_p2**, mediante la opción "***Data Import***" del menú ***Server*** de **MySqlWorkbench** a partir de dicho fichero "**dumpBDAprac2.sql**", disponible junto con este enunciado y el diccionario de datos en el Moodle de la asignatura. Al ejecutar esta opción se mostrará una ventana en la que habrá que seleccionar la opción "Import from Self_Contained File" indicando la ubicación del fichero dumpBDAprac2.sql, y a continuación hacer clic en el botón "Start Import" que figura en la parte inferior derecha de la ventana.

b) Realizar ingeniería inversa desde MySqlWorkbench (menú Database, opción "Reverse Engineer") y analizar el diagrama obtenido. ¿Por qué no hay conexiones entre las tablas de la base de datos?

c) Una vez creada la base de datos, ejecutar la siguiente sentencia al catálogo de MySql para hacerse una idea aproximada del volumen de datos (filas y ocupación en disco) de cada una de las tablas que componen la base de datos.

```
SELECT
    TABLE_NAME,
    TABLE_ROWS AS registros_aproximados,
    ROUND((DATA_LENGTH + INDEX_LENGTH)/1024/1024, 2) AS tamano_MB
FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_SCHEMA = 'bda_p2'
ORDER BY TABLE_NAME ASC;
```

El catálogo de MySql se compone de varias tablas o vistas, algunas de las que pueden resultar interesantes en el desarrollo de la práctica son: TABLES, TABLE_CONSTRAINTS, KEY_COLUMN_USAGE, STATISTICS, COLUMNS.

d) Comprobar que de las tablas creadas al cargar la base de datos hay una que tiene definida la PK.

```
SELECT *
FROM INFORMATION_SCHEMA.TABLE_constraints
WHERE TABLE_SCHEMA = 'bda_p2';
```

Intentar eliminar la restricción de clave principal de dicha tabla con la siguiente sentencia:

```
ALTER TABLE tabla DROP PRIMARY KEY;
```

Averiguar por qué no se puede eliminar y qué pasos hay que realizar para eliminar la clave primaria sin eliminar previamente la tabla.

Apartado 2. Estudio de planes de consulta e índices.

a) Crea una consulta SQL que muestre cuántos empleados activos (date_of_termination = '9999-01-01') trabajan **en cada ciudad de Alemania** (Germany) ordenando el resultado por número de empleados descendentemente. Plantea al menos **dos soluciones distintas** para esta consulta, una de ellas utilizando el operador **EXISTS**, de forma que se aprecien diferencias significativas entre ellas respecto al coste y el tiempo de ejecución.
b) Para cada una de las soluciones, indica el plan de ejecución, si se ha podido obtener el resultado y a qué conclusión llegas.
c) Dado que se ha importado la base de datos con algunas tablas voluminosas puede ser recomendable utilizar ANALYZE TABLE u OPTIMIZE TABLE. Averigua si conviene ejecutar alguna de ellas.
d) Tras ello, compara nuevamente los planes de consultas de ambas soluciones e indica las diferencias.

Apartado 3. Crear Claves Primarias.

a) Crear las claves principales mediante el fichero script *Apartado3_CrearClavesPrimarias.sql*.
b) Estudiar el plan de consulta de las sentencias anteriores del apartado 2a comparando costes con los obtenidos en dicho apartado.
c) Analizar si el coste es igual o similar al obtenido antes de la creación de las PK.
d) ¿Por qué access_type: "ALL" (Full Table Scan) en alguna de las tablas pese a tener PK?

Apartado 4. Crear Claves Foráneas.

a) Crear las claves foráneas mediante el fichero script *Apartado4_CrearClavesForaneas.sql,* y nuevamente estudiar el plan de consulta, comparando costes con los dos apartados anteriores. Analizar la diferencia o semejanza en costes estimados entre la primera consulta y la de EXISTS.
b) Qué índices se crean automáticamente al añadir las claves foráneas (consultar las tablas del catálogo de MySql).

c) Comprueba la información que se obtiene ejecutando consultas a las tablas del catálogo de las PK, FK y de los índices de las tablas de la base de datos (TABLE_CONSTRAINTS, KEY_COLUMN_USAGE, STATISTICS, etc.).
d) Analiza la información del resultado que se obtiene consultando la tabla statistics (TABLE_NAME, INDEX_NAME, NON_UNIQUE, SEQ_IN_INDEX, CARDINALITY).

Apartado 5. Crear índices para optimizar la consulta anterior.

a) Analizar la cardinalidad y selectividad de los atributos que figuran en la condición WHERE mediante consultas Sql a la tabla del atributo y a la tabla Statistics. Ejemplo para cardinalidad:

```
-- cardinalidad del campo employee.date_of_termination
SELECT COUNT(DISTINCT date_of_termination)/(SELECT CARDINALITY
                                            FROM information_schema.STATISTICS
                                            WHERE TABLE_SCHEMA = 'bda_p2'
                                              AND table_name = 'employee') t
FROM employee;
```

b) Crear índices para cada uno de los atributos que figuran en el WHERE, determinando cómo afectan al coste del plan de ejecución en las soluciones anteriores a la consulta.
c) Estudiar los planes de consultas ejecutando VISUAL EXPLAIN, EXPLAIN y EXPLAIN ANALYZE FORMAT=TREE SELECT de cada una de las consultas. Analiza los resultados e indica alguna conclusión.
d) Con los dos índices creados, probar la utilización de INDEX HINTS (USE | IGNORE |FORCE INDEX) y STRAIGHT_JOIN para analizar si se produce alguna variación en el plan y coste de ejecución.

Apartado 6. Optimizar la siguiente consulta.

a) Describir en lenguaje natural que se obtiene con la siguiente consulta

```
SELECT e.emp_no, concat(first_name, ' ', middle_names, ' ', last_name),
AVG(salary) AS avg_salary
FROM employee e inner join salary s on e.emp_no = s.emp_no
where year(date_of_birth) = 1990
  and e.emp_no in (select emp_no
    from city_emp ce inner join city t on ce.city_id = t.id
    inner join country c on t.country_id = c.country_id
    where continent <> 'Europe' OR continent <> 'Africa'
and (c.population*1.1) > 1000000 and district like '%a')
GROUP BY e.emp_no
HAVING AVG(salary) > 50000;
```

b) Eliminar los índices creados en el apartado anterior, manteniendo solo claves primarias y foráneas.
c) Crear los índices que permitan optimizar el coste de las consultas, analizando plan de consulta y coste para cada uno de los casos, justificando que solución es la mejor.
d) Transformar la consulta como se estime necesario para optimizar su plan de ejecución.

Apartado 7. Cuando el optimizador de MySQL omite índices.

a) ¿Por qué el optimizador de MySQL puede decidir no usar un índice, aunque dicho índice exista en la tabla? Explica qué significa cada uno de los siguientes motivos, pon un ejemplo de consulta ligera en SQL para cada caso y propón una posible solución:

1. El índice no es selectivo (baja selectividad).

2. La consulta devuelve un porcentaje muy grande de la tabla.
3. Uso de operaciones sobre columnas indexadas.
4. Funciones en columnas indexadas .
5. El índice no cubre todas las columnas necesarias.
6. Uso de OR en las condiciones de la consulta.
7. Consultas con ORDER BY y LIMIT.
8. Uso de LIKE con comodín al principio

b) ¿Qué entiende MySQL por selectividad de un índice y cómo influye en la decisión de usarlo o no? ¿En qué situaciones un escaneo completo de tabla (FULL TABLE SCAN) puede ser más eficiente que usar un índice?

Apartado 8: Estudio de índices en actualizaciones

Vamos a cuantificar el impacto del rendimiento en escrituras al añadir múltiples índices.

a) Ejecuta comprobando el tiempo de ejecución de varias operaciones de actualización tomando nota de los tiempos de ejecución y planes en su caso. Por ejemplo:

```
-- Copia la tabla de salary
 CREATE TABLE salary_test LIKE salary;
-- Inserta 10.000 registros
INSERT INTO salary_test SELECT * FROM salary WHERE emp_no < 40000 LIMIT
10000;
-- Quedarse solamente con la clave primaria
SHOW INDEX FROM salary_test;  -- Solo PRIMARY

-- INSERT masivo
INSERT INTO salary_test (emp_no, salary, from_date, to_date)
SELECT emp_no, salary+1000, from_date, to_date
FROM salary WHERE emp_no BETWEEN 12000 AND 22000;

-- UPDATE masivo
UPDATE salary_test SET salary = salary * 1.1 WHERE emp_no > 10000;

-- DELETE masivo
delete from salary_test where emp_no > 20000;
```

b) Ejecutar TRUNCATE TABLE sobre la tabla salary_test y crear cuatro índices: (from_date), (to_date), (salary), (to_date, emp_no, salary). Después ejecutar las mismas operaciones que en el apartado anterior para contrastar tiempos y costes.

Apartado 9. "Desnormalización: de JOINs Lentos a Lecturas Instantáneas"

Objetivo: Comparar el rendimiento de JOINs en tablas normalizados vs tablas desnormalizadas, midiendo impacto real. Una consulta habitual es la de obtener datos de empleados junto con el país de su trabajo actual y el número de países diferentes en los que ha trabajado.

a) Especificar una consulta sql para obtener la información requerida. Anotar tiempo y coste.
b) Desnormalizar: crear y cargar las nuevas tablas con la desnormalización elegida para realizar la consulta objetivo. Para evitar cambios en las tablas existentes crear la tabla desnormalizada mediante:

```
CREATE TABLE employee_desnor LIKE employee;
INSERT INTO employee_desnor SELECT * FROM employee;
-- Añadir los atributos de la desnormalización a la tabla employee_desnor
```

```
ALTER TABLE employee_desnor ADD <atributo>;
```

c) Simular actualización mediante sql para que los datos sean consistentes tanto de las tablas normalizadas como los de la no normalizada. Analizar los planes y tiempos de ejecución.

d) Crear los triggers necesarios en la base de datos para automatizar las modificaciones de datos en la tabla normalizada. Comparar costes y tiempos d ejecución frente a las tablas normalizadas.

Apartado 10. Mejora del rendimiento de consultas mediante particionamiento en MySQL.

a) Implementar particionamiento por rangos de fechas (from_date) en tablas con grandes volúmenes de datos (salary). Crear una nueva tabla de nombre salary_part con particiones cada cinco años. solicitado y posteriormente insertar las filas en esta tabla particionada a partir de la tabla salary mediante INSERT INTO … SELECT

b) Comparar el rendimiento de consultas entre tablas particionadas y no particionadas, utilizando métricas de tiempo y plan de ejecución. Utilizar para ello la tabla salary y salary_part. Interpretar el resultado de EXPLAIN para comprender cómo el motor de MySQL accede a las particiones. Consultas:

   1. Obtener los datos de salarios entre las fechas (from_date) comprendidas entre '2010-01-01' y '2025-08-01'.
   2. Obtener los datos de salarios de los años 2025 y 2026.
   3. Obtener los datos de los salarios superiores a 70000.
   4. Obtener los datos de los salarios superiores a 70000 en los años 3035 y 2026.

Apartado 11. Resuelve las siguientes consultas SQL.

Resuelve en SQL las siguientes consultas:

1. Obtener, sin duplicados, el nombre de los países que tienen trabajadores en activo. Plantea una solución con distinct y otra con EXISTS. Analizar costes y tiempos de ejecución.
2. Obtener empleados (sus tres campos de nombre separados por un espacio en blanco) que hayan trabajado en Sevilla y en Cádiz (en las dos).
3. Para aquellos empleados que nunca hayan trabajado en Europa (Europe) obtener aquel que haya trabajado en más ciudades no europeas con población inferior a 1000 habitantes. Para obtener el máximo plantea una solución con Having con >=ALL y otra sin Having pero con LIMIT.

Apartado 12. Establecer conclusiones generales de todo el trabajo.